Chapter 12 — Distributed systems overview

Single-node disks cannot hold petabyte corpora

Learning objectives

- By the end of this part

Why distributed systems

- Distributed systems sit at the heart of scalable data management
- They enable data to be stored, processed
- Chapter 9 introduced streaming collection from an acquisition angle

Key characteristics

- Distributed systems aim for high availability, fault tolerance, and horizontal scalability
- Core characteristics include data partitioning across nodes

Example 12.7 — HDFS Partitioning and Replication

- Example 12.7 — hands-on module
- Example 12.7 walks through HDFS blocks as a distributed-file pattern
- HDFS splits large files into blocks
- Explore the chapter example module
- View files: `modules/chapter12/example7/`

Example 12.8 — Cassandra for Multi-Datacenter Writes

- Example 12.8 — hands-on module
- Example 12.8 positions Cassandra for write-heavy global apps
- Apache Cassandra is a distributed NoSQL store built for high write throughput and
- Explore the chapter example module
- View files: `modules/chapter12/example8/`

Example 12.9 — Kafka for Real-Time Event Streams

- Example 12.9 — hands-on module
- Example 12.9 uses Kafka as the streaming backbone
- Apache Kafka ingests and buffers event streams with durable
- Explore the chapter example module
- View files: `modules/chapter12/example9/`

Patterns that recur

- Together, HDFS, Cassandra
- File blocks, write-optimized tablet stores

Takeaways

- Fault tolerance
- HDFS shows block placement and replica recovery
- Cassandra targets multi-datacenter write scale
- Kafka provides durable partitioned logs for real-time consumers
- These patterns prepare the CAP and consistency discussion that follows

Next

- Complete the quiz for this part
- The next part covers the CAP theorem, sharding and replication patterns